Anexo F: Lenguajes de Marcas en la Web

HTML

HTML 1

HTML 1.0 surgió en el CERN de la mano de Tim Berners-Lee como un perfil simplificado derivado de SGML (Standard Generalized Markup Language). En esta fase no existía un organismo oficial de estandarización ni una especificación formal, sino un documento descriptivo con algo menos de veinte elementos orientados exclusivamente a la estructuración jerárquica de hipertexto basado en texto plano (`<html>`, `<head>`, `<body>`, `<h1>`-`<h6>`, `<p>`, `<a>`). La arquitectura carecía de capacidades de maquetación, hojas de estilo o lógica ejecutable. El procesamiento de archivos multimedia se realizaba de forma externa al navegador, ya que el motor de renderizado no incorporaba un parser determinista para imágenes alineadas en el flujo de texto, y tampoco existía un subsistema para la transferencia de datos desde el cliente hacia el servidor.

HTML 2.0 (1995)

La primera estandarización formal llegó con HTML 2.0, publicado por el IETF (Internet Engineering Task Force) bajo el RFC 1866. Esta especificación introdujo la integración nativa del elemento para el renderizado de gráficos dentro del flujo de texto (inline graphics) y formalizó el subsistema de formularios mediante los elementos `<form>`, `<input>`, `<select>` y `<textarea>`. La transferencia de información entre cliente y servidor se estructuró a través de los métodos HTTP GET y POST empleando la codificación `application/x-www-form-urlencoded`. Asimismo, se incorporó el soporte inicial para entidades de caracteres SGML, extendiendo la capacidad del lenguaje para representar símbolos fuera del estándar ASCII.

HTML 3.2 (1997)

El W3C (World Wide Web Consortium) asumió la estandarización del lenguaje con la publicación de HTML 3.2 como Recomendación Oficial en enero de 1997, tras descartar el borrador inestable de HTML 3.0. Esta versión introdujo el modelo de tablas (`<table>`, `<tr>`, `<td>`), que aunque fue diseñado para representar datos tabulares, comenzó a utilizarse masivamente como mecanismo de maquetación bidimensional en el DOM. HTML 3.2 integró atributos de presentación visual directa dentro del marcado (`<font>`, `<center>`, atributos de alineación y color), además de habilitar la ejecución de código en el cliente mediante el elemento `<applet>` para bytecode de Java y las bases para la inclusión de scripts.

HTML 4.01 y XHTML 1.0 (1999–2000)

HTML 4.01 se aprobó en 1999 para establecer la separación estricta entre la arquitectura del marcado y la capa de presentación, desaconsejando las etiquetas visuales directas en favor

de las Hojas de Estilo en Cascada (CSS). La sintaxis se dividió en tres Document Type Definitions (DTD) distintos: Strict, Transitional y Frameset. Esta versión consolidó el soporte para caracteres Unicode, atributos de accesibilidad (alt, title) y el elemento `<iframe>`. En el año 2000, el W3C reformuló el estándar bajo XHTML 1.0, supeditando el código a las reglas sintácticas estrictas de XML 1.0, lo que exigía un árbol de documento perfectamente formado con cierre explícito de etiquetas vacías (`<br />`), sensibilidad a mayúsculas y entrecomillado estricto en todos los atributos.

HTML 5 (2014)

Desarrollado inicialmente por el WHATWG desde 2004 y adoptado como Recomendación Oficial por el W3C en 2014, HTML5 reemplazó el modelo DTD/SGML por un algoritmo de parseo determinista a nivel de motor de renderizado mediante el doctype `<!DOCTYPE html>`. Esta actualización introdujo elementos semánticos para definir la arquitectura de información (`<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<footer>`), así como una pipeline multimedia nativa mediante los elementos `<video>` y `<audio>`. En el ámbito gráfico incorporó la etiqueta `<canvas>` con API de contexto 2D y soporte para WebGL con aceleración por GPU, junto con la integración directa de SVG en el DOM. Los formularios añadieron validación nativa sin JavaScript a través de tipos de entrada específicos (email, date, number), mientras que el entorno de ejecución se amplió con APIs para almacenamiento local (localStorage, IndexedDB), concurrencia (Web Workers), comunicación bidireccional (WebSockets) y control del historial de navegación.

XHTML vs. HTML

La diferencia técnica fundamental entre ambos estándares (HTML y XHTML) reside en el motor de procesamiento que interpreta el documento. HTML opera mediante un algoritmo determinista (partiendo de los mismos datos de entrada, siempre produce exactamente el mismo resultado y sigue el mismo camino lógico paso a paso) que asimila errores de sintaxis (como etiquetas sin cerrar, anidamientos cruzados o atributos mal formados) y reconstruye un árbol DOM válido sin interrumpir la experiencia de usuario.

XHTML, al ser una aplicación directa de XML 1.0, exige que el documento sea estrictamente bien formado (well-formed). Si el parser XML encuentra una sola violación de la sintaxis, interrumpe el proceso de renderizado y despliega un error fatal en pantalla (Yellow Screen of Death).

Aspecto / Característica	HTML (Estándar Flexible)	XHTML (Estándar XML Estricto)
SEMEJANZAS		

Aspecto / Característica	HTML (Estándar Flexible)	XHTML (Estándar XML Estricto)
Propósito principal	Estructurar y presentar contenido en la Web.	Estructurar y presentar contenido en la Web.
Elementos base	Usa etiquetas como `<a>`, `<p>`, `<div>`, `<table>`, `<img>`.	Usa las mismas etiquetas base heredadas de HTML.
Integración tecnológica	Se combina con CSS para estilos y JavaScript para interactividad.	Se combina con CSS para estilos y JavaScript para interactividad.
Compatibilidad	Interpretado de forma nativa por todos los navegadores web.	Interpretado por navegadores web (soporte parcial en versiones antiguas).
DIFERENCIAS		
Estándar base	Basado originalmente en SGML / Estándar propio (WHATWG/W3C).	Basado estrictamente en **XML**.
Tolerancia a errores	**Permisivo:** El navegador intenta corregir el código mal escrito para mostrarlo.	**Estricto:** Un error de sintaxis detiene el procesamiento o falla el renderizado.
Cierre de etiquetas	Opcional en etiquetas vacías (ej. ` `, `<img>`, `<input>`).	**Obligatorio** en todas las etiquetas (ej. ` `, `<img />`, `<input />`).
Mayúsculas / Minúsculas	Indiferente (acepta `<DIV>`, `<div>` o `<DiV>`).	**Obligatorio escribir todo en minúsculas** (solo `<div>`).
Comillas en atributos	Opcionales en valores simples (ej. `width=100`).	**Obligatorias siempre** (ej. `width="100"`).
Minimización de atributos	Permitida (ej. `<input checked>` o `<option selected>`).	**Prohibida** (ej. `<input checked="checked" />`).

Aspecto / Característica	HTML (Estándar Flexible)	XHTML (Estándar XML Estricto)
Anidamiento de etiquetas	El navegador suele corregir desórdenes leves.	**Estricto:** Deben cerrarse exactamente en el orden inverso al que se abrieron.
Declaración `<html>`	Etiqueta simple: `<html>`.	Requiere incluir el namespace XML: `<html xmlns="[http://www.w3.org/1999/xhtml](http://www.w3.org/1999/xhtml)">`.

¿Entonces cuál es la diferencia fundamental o de concepto entre ambos?

Para entenderlo de forma sencilla, imagina que el HTML normal es como una conversación informal entre amigos: si te comes una palabra o te explicas a medias, la otra persona igual te entiende porque adivina tu intención.

En cambio, XHTML estricto es como un contrato legal o un programa informático: las reglas son fijas y no hay espacio para interpretaciones. Si falta una coma o un cierre, el sistema se detiene y rechaza el documento.

En la realidad cuando una página web se conecta con un sistema de gestión (como el de una base de datos de nuestro inventario de componentes), ya no la está leyendo solo una persona, sino un programa automático.

Para esto hay una serie de conceptos que garantizan que todo funcione sin errores:

- **El Analizador Sintáctico** (Parser): Es el programa informático encargado de leer y traducir tu página web para que la base de datos la entienda. En HTML: si el navegador encuentra una etiqueta mal cerrada (por ejemplo, abres una negrita `<b>` pero nunca la cierras), el navegador "hace trampa" e intenta adivinar dónde querías cerrarla. Sin embargo, en XHTML los analizadores de bases de datos no adivinan. Si ven un error, se rompen o rechazan los datos. Al exigir una sintaxis estricta, te aseguras de que el sistema nunca falle por culpa de un malentendido de código.
- **Consultas precisas con XPath**: Imagina que necesitas buscar automáticamente en tu base de datos el precio de todos los productos que están dentro de una tabla. Con XHTML, al ser un documento ultra ordenado (como una estructura de árbol perfecta), puedes usar una herramienta técnica llamada XPath para "navegar" exactamente hasta el dato que buscas sin miedo a que una etiqueta mal colocada desvíe la búsqueda.
- **Validación previa**: Como XHTML se basa en las reglas del formato XML, los sistemas de inventario pueden pasar el código por un filtro automático (un esquema de

validación) antes de guardarlo en la base de datos. Si el texto tiene un error de estructura, el sistema lo bloquea automáticamente, evitando que rompa la página web más adelante.

¿Justificas la utilidad de la sintaxis estricta de XHTML cuando se integran páginas web con sistemas de gestión de información, inventarios y bases de datos?

Sí, se justifica plenamente. Aunque hoy en día HTML5 es la norma general por su flexibilidad, la sintaxis estricta de XHTML es una gran opción cuando una página web necesita conectarse de forma directa con bases de datos u otros programas informáticos.

La razón principal es que XHTML no permite errores en la estructura del código. Mientras que el HTML convencional intenta corregir sobre la marcha cualquier etiqueta mal cerrada u otros errores de sintaxis para que la página web se vea bien en pantalla, XHTML exige una sintaxis más limpia.

Al basarse exactamente en las reglas de XML (estándar creado por el W3C para almacenar, organizar e intercambiar datos), XHTML garantiza que la información de la web tenga la misma estructura limpia, jerárquica y ordenada que maneja una base de datos o un sistema de inventario.

¿Por qué un documento XHTML mal formado puede fallar al conectarse con una base de datos o sistema de inventario, mientras que HTML5 suele ser más permisivo con los errores del desarrollador?

Un sistema de inventario o una base de datos que procesa datos web de forma automatizada necesita leer estructuras exactas para saber qué es un precio, qué es un código de producto o a qué categoría pertenece un stock. Si la página utilzara HTML flexible, un script de extracción podría interpretar mal los límites de las etiquetas debido a un error del código HTML, terminando por guardar datos corruptos, sin que nadie lo note. Con XHTML, la exigencia de estar perfectamente formado hace que: si el código tiene un fallo, el sistema automatizado lo detecta y frena la operación de la base de datos antes de que un error ocurra.

Ejemplos de por qué XHTML es más estricto que HTML

- Minúsculas obligatorias: Todas las etiquetas y atributos deben escribirse en minúsculas (`<body>`, `<p>`). HTML permite mayúsculas o combinadas (`<BODY>`, `<P>`).
- Cierre obligatorio de etiquetas vacías: Elementos que no tienen etiqueta de cierre propia (como `<br>`, `<img>`, `<input>`) deben cerrarse a sí mismos agregando espacio y barra al final (`<br />`, `<img />`).
- Comillas obligatorias en atributos: Todos los valores de atributo deben ir entre comillas dobles o simples. En HTML es válido src=foto.jpg, pero XHTML exige `src="foto.jpg"`.

- Prohibición de atributos abreviados: Los atributos booleanos no se pueden minimizar. En lugar de escribir simplemente `checked` o `disabled`, XHTML requiere `checked="checked"` o `disabled="disabled"`.
- Anidamiento estricto (FIFO/LIFO): Las etiquetas deben cerrarse exactamente en el orden inverso al que se abrieron. La secuencia `<b><i>texto</b></i>` es aceptada por la tolerancia de HTML, pero XHTML exige `<b><i>texto</i></b>`.

¿Qué problemas surgirían al mantener la web del almacén si aplicaras los estilos visuales directamente con atributos HTML incrustados en lugar de utilizar un archivo CSS externo?

Mantener una web de almacén aplicando estilos visuales directamente con atributos incrustados en lugar de utilizar un archivo CSS externo generaría problemas graves de mantenimiento.

- Dispersión absoluta del código: Cualquier cambio general de diseño obligaría a buscar y modificar manualmente cada etiqueta en muchas páginas diferentes. Totalmente inostenible.
- Peso de cada documento HTML: Al mezclar el HTML con el CSS la web crece innecesariamente, lo que ralentiza la velocidad de carga y dificulta la lectura automatizada por parte de los sistemas de inventario.
- Adaptar la interfaz de forma responsiva: Al no contar con hojas de estilo centralizadas es imposible su adaptación a diferentes tamaños de pantalla.

Ventajas que aporta separar el HTML del CSS

Separar la estructura HTML de la presentación visual en un archivo CSS optimiza el desarrollo web al centralizar los estilos en un único archivo, lo que simplifica el mantenimiento global y elimina etiquetas obsoletas para lograr un código mucho más limpio. En términos de rendimiento, el navegador almacena el diseño en caché tras la primera visita y reduce drásticamente el peso de transferencia en la red. Además, mejora la accesibilidad y el posicionamiento SEO al ofrecer una jerarquía semántica clara para lectores de pantalla y buscadores, permitiendo adaptar el contenido a cualquier dispositivo mediante media queries. Por último, otorga una gran flexibilidad al permitir rediseñar interfaces por completo sin tocar el código base y fomenta un trabajo en equipo más eficiente entre desarrolladores y diseñadores. ´

Anexo F: Árbol DOM de la Página principal

```
HTML
  ├── HEAD
  │     ├── META (charset)
  │     ├── META (viewport)
```


META (author)
META (description)
META (copyright)
META (keywords)
TITLE
LINK (stylesheet)
LINK (icon)
LINK (preconnect)
LINK (preconnect)
LINK (google fonts)
BODY (#top)
HEADER
IMG (.imgHeader)
H1
A (.headerCastor)
MAIN
SECTION (.intro)
IMG
SECTION (.banner)
H1
SECTION (.seccion-listado)
BR
P (.seccion-listado-texto)
UL (.menu)
LI (.menu-lista) (x9)
A
SPAN
SPAN (.arrow)
SPAN (.menu-lista-span)
IMG
FOOTER
SECTION (.footer-content)
ARTICLE (.footer-col)
H1
P
ARTICLE (.footer-col)
NAV
UL
LI (x5)
A
ARTICLE (.footer-col)
NAV
DETAILS

```
        │    │              ├── SUMMARY
        │    │              └── UL
        │    │                   └── LI (x8)
        │    │                        └── A
        │    └── ARTICLE (.footer-col)
        │         └── IFRAME (.footer-col-iframe)
        ├── P (.footer-copy)
        └── A (.btnArriba)
```